



WELCOME

Data Modeling in MongoDB

Datev Coding Day



Hi, I'm Tobias Joschko!

You can find me on LinkedIn.



**Tobias
Joschko**

- Solutions Architect at MongoDB
- Software Developer
- Lecturer for Software Development

What this session **ISN'T:**

3-hour long presentation

Abstract and detached
from reality

Persuading you to use
MongoDB



What this session **IS:**

Engaging — you will be
doing most of the work!

Applicable and transferable
to your own projects

Open discussions about the
pros and cons of the
techniques we'll see

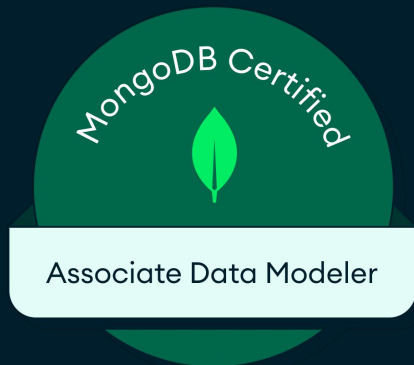


Hands-on sessions

- No stupid questions
- Actively discuss
- Be respectful

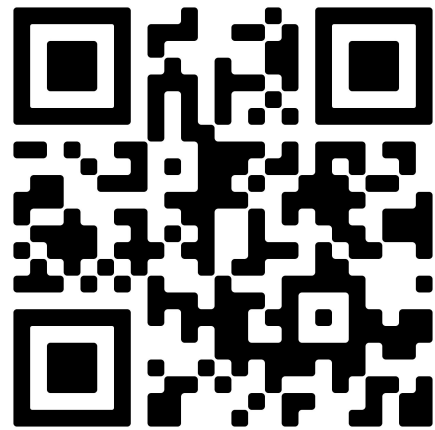


Exclusive Offer



New Associate Data Modeler Certification

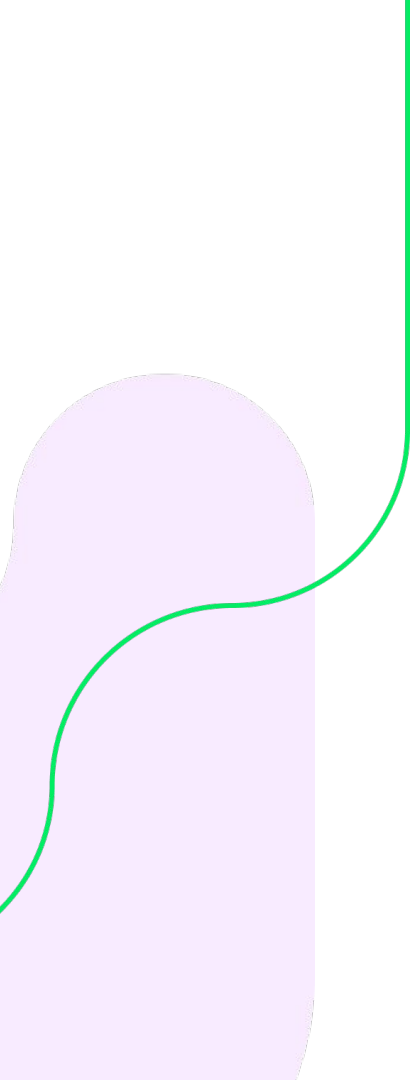
Complete the learning path and unlock a 75% discount. Take the certification exam for only \$37.50.



Start here to prepare for
the exam. Register now!



Data modeling is the
process of figuring out how
to represent and store data

A decorative graphic on the left side of the slide. It consists of a light purple, rounded rectangular shape at the bottom, with a thin green line curving upwards from its top right corner and extending towards the top left of the slide.

Why is data
modeling
important?

A good
data model
leads to:



Better performance



Simpler implementation



Lower costs



Better adaptability





Data modeling in
MongoDB is often
overlooked and done
inefficiently

“NoSQL”

“Schemaless”

Modeled using RDBMS
techniques



Data modeling in
MongoDB is often
overlooked and done
inefficiently

“NoSQL”

“Schemaless”

Modeled using RDBMS
techniques



Books		

Authors		

--	--

```
SELECT * FROM Books, Authors, Reviews
WHERE Books.author_id = Authors.id
AND Books.id = Reviews.book_id
```

--	--



“NoSQL”



~~“NoSQL”~~

“Non-relational”



~~“NoSQL”~~

~~“Non-relational”~~

Non-tabular



Non-tabular databases

Document databases

Key-value databases

Wide-column stores

Graph databases



Non-tabular databases

Document databases

Key-value databases

Wide-column stores

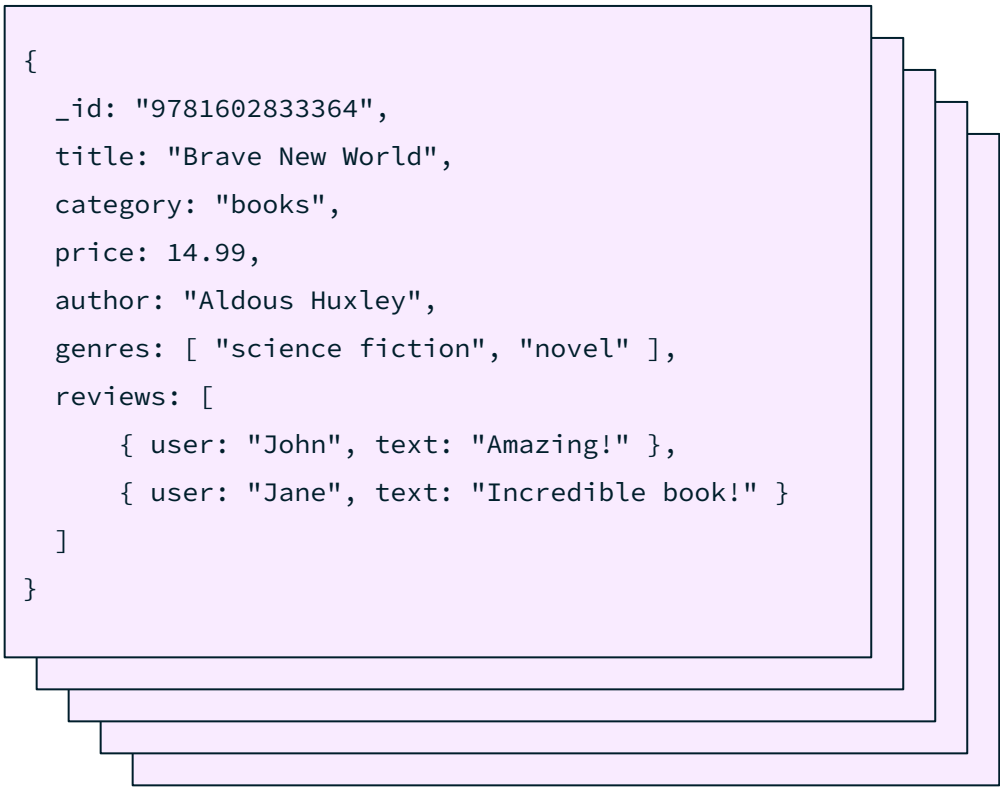
Graph databases

A “book” document

```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  author: "Aldous Huxley",
  genres: [ "science fiction", "novel" ],
  reviews: [
    { name: "John", text: "Amazing!" },
    { name: "Jane", text: "Incredible book!" }
  ]
}
```



“Books” collection

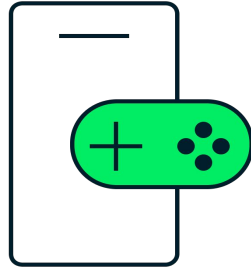


```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  author: "Aldous Huxley",
  genres: [ "science fiction", "novel" ],
  reviews: [
    { user: "John", text: "Amazing!" },
    { user: "Jane", text: "Incredible book!" }
  ]
}
```





Data modeling quiz time!



Answer in Slido



Answer the
question in
Slido!

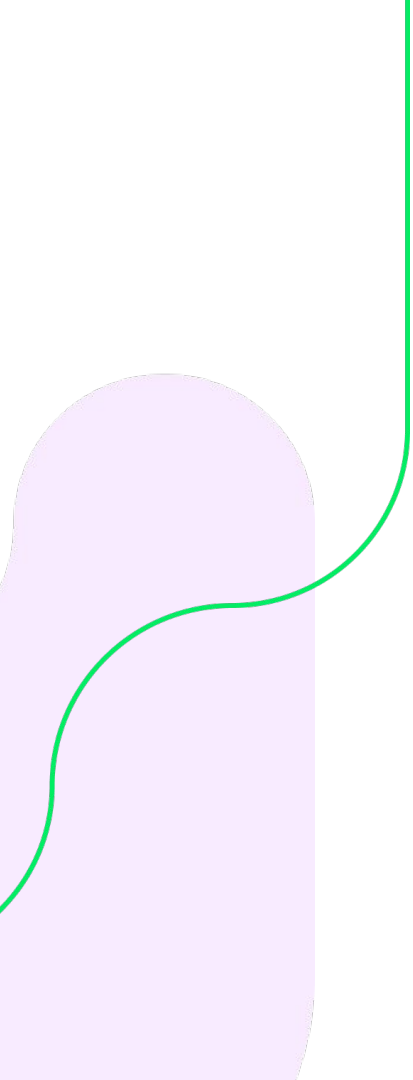
Why is data modeling
in MongoDB often
overlooked or done
ineffectively?

“NoSQL”

“**Schemaless**”

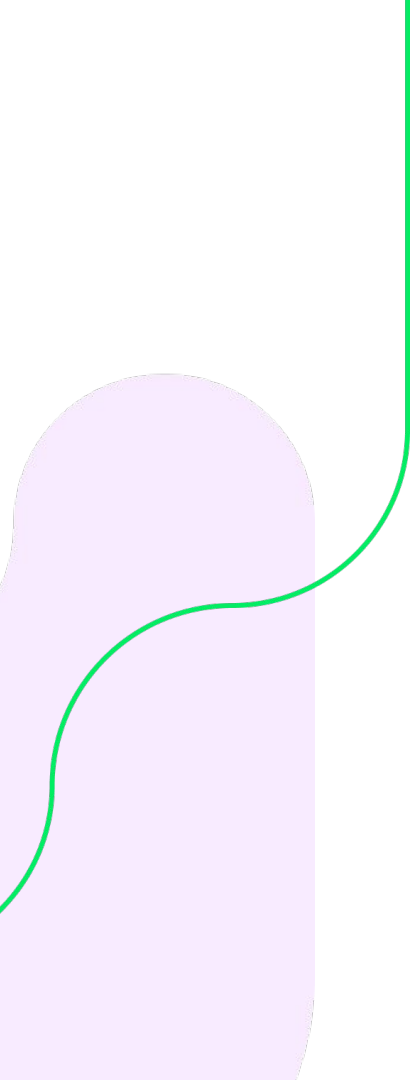
Modeled using RDBMS
techniques





MongoDB is not
a JSON store.

MongoDB is not
schemaless.

A decorative graphic on the left side of the slide consists of a light purple rounded rectangle and a thin green line that curves around its top and right edges.

MongoDB has a
flexible schema
model.



Flexible schema: unlocking developer velocity



Avoids the need to update
ORM class mappings and
recompile programming
language classes



Schema changes don't lock
the database or cause
performance degradation
while tables are altered



Breaks down complex
inter-group dependencies
and expensive coordination
before new code is released



```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  author: "Aldous Huxley"
}
```

New field

```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  author: "Aldous Huxley"
  publisher: "Langdon",
  schemaVersion: 2
}
```



Why is data modeling
in MongoDB often
overlooked or done
ineffectively?

“NoSQL”

“Schemaless”

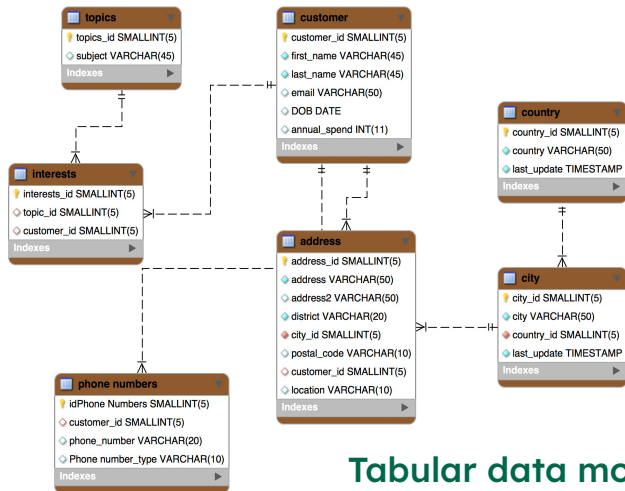
**Modeled using RDBMS
techniques**



Document Data Model

Related data contained in a single, rich document

```
{
  "_id" : ObjectId("5ad88534e3632e1a35a58d00"),
  "name" : {
    "first" : "John",
    "last" : "Doe" },
  "address" : [
    { "location" : "work",
      "address" : {
        "street" : "16 Hatfields",
        "city" : "London",
        "postal_code" : "SE1 8DJ"},
      "geo" : { "type" : "Point", "coord" : [
        -0.109081, 51.5065752]}}],
  +   {...}
],
  "dob" : ISODate("1977-04-01T05:00:00Z"),
  "retirement_fund" : NumberDecimal("1292815.75")
}
```



Tabular data model

Related data split across multiple records and tables

Contrasting data models

Data Modeling Methodology for MongoDB



Prerequisite: Collect the system requirements.

1. Identify the database entities and their properties.
2. Determine the database workloads. Write down the common operations, their frequency and criticality.
3. Identify the relationships between entities. Choose between embedding and referencing.
4. Apply appropriate design patterns if necessary.

Data Modeling Methodology for MongoDB



Prerequisite: Collect the system requirements.

1. Identify the database entities and their properties.
2. Determine the database workloads. Write down the common operations, their frequency and criticality.
3. Identify the relationships between entities. Choose between embedding and referencing.
4. Apply appropriate design patterns if necessary.



Answer the
question in
Slido!



Library Management

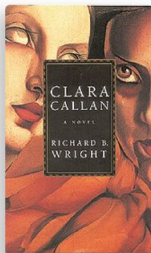
The System We're Designing
Today



Library Management System

Daft Platypus  Admin 

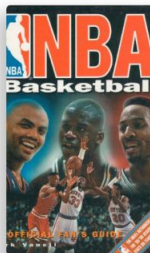
Find books...*



Clara Callan: A novel

by [Richard Bruce Wright](#)

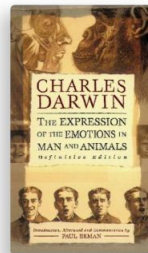
Giller Prize Winner 2001. Richard B. Wright. A Phyllis Bruce Book.



NBA: The Official Fan's Guide

by [Mark Vancil](#)

A vibrantly illustrated and informative guide that offers up-to-the-minute reporting on the 1995-96 season, comprehensive histories of classic team



Expression of the Emotions In Man and Anim

by [Charles Darwin](#)

Charles Darwin. Includes Bibliographical References And Index.

Functional Requirements



The system should allow **readers** to:

- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, synopsis — available physical copies, and reviews from readers.
- R3. Make a book reservation if there are physical copies available. Book reservations expire after 12 hours.
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.

The system should allow **librarians** to:

- L1. See a list of all book reservations, including their expiration time.
- L2. See a list of all borrowed books, including their due date.
- L3. Filter reservations and borrowed books by user.
- L4. Lend books to readers.
- L5. Mark books as returned.

Library Data

- A. The library has **10k active readers**.
- B. The library has about **250k books**.
- C. Readers will consult the library website on a **weekly basis** and borrow books **every month**.
- D. Most readers **reserve** and **borrow** up to **3 books** every month.
- E. **Reservations** can be made online, but books can only be **borrowed** on-site.
- F. Readers rarely read more than **5 book reviews**.
- G. Readers browse about **25 books** before reserving.
- H. Readers leave **reviews for 1 in 5 books** they read.

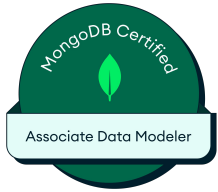
Performance Requirements



- P1. The book details page that shows book availability and recent reviews should load in less than **300ms**.
- P2. Readers should be able to reserve a book in less than **3 seconds**.
- P3. Librarians should be able to lend a book or mark a book as returned in less than **3 seconds**.
- P4. Readers should be able to see the list with all of their reservations and borrowed books in less than **500ms**.

Data Modeling Methodology





Entities

- Identify the entities
- Quantify the data size

Workloads

- Quantify the operations
- Qualify the operations

Relationships

- Identify the relationships
- Quantify the relationships
- Embed or reference

Patterns

- Identify the pattern
- Apply the pattern



Step 1.

Identify the entities

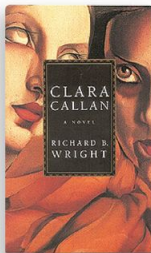
- What are the main entities that are used in this system and should be stored in the database?
- What are the properties of these entities? For example, every reservation has a due date, every review — a score, every book — a title.



Library Management System

Daft Platypus  Admin 

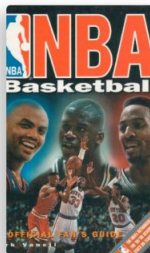
Find books...*



Clara Callan: A novel

by [Richard Bruce Wright](#)

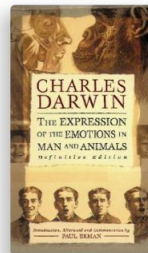
Giller Prize Winner 2001. Richard B. Wright. A Phyllis Bruce Book.



NBA: The Official Fan's Guide

by [Mark Vancil](#)

A vibrantly illustrated and informative guide that offers up-to-the-minute reporting on the 1995-96 season, comprehensive histories of classic team



Expression of the Emotions In Man and Anim

by [Charles Darwin](#)

Charles Darwin. Includes Bibliographical References And Index.

Functional Requirements



The system should allow **readers** to:

- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, synopsis, available physical copies, and reviews from readers.
- R3. Make a book reservation if there are physical copies available. Book reservations expire after 12 hours.
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.

The system should allow **librarians** to:

- L1. See a list of all book reservations, including their expiration time.
- L2. See a list of all borrowed books, including their due date.
- L3. Filter reservations and borrowed books by user.
- L4. Lend books to readers.
- L5. Mark books as returned.

Entities

Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN

Entities

Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN

Entities

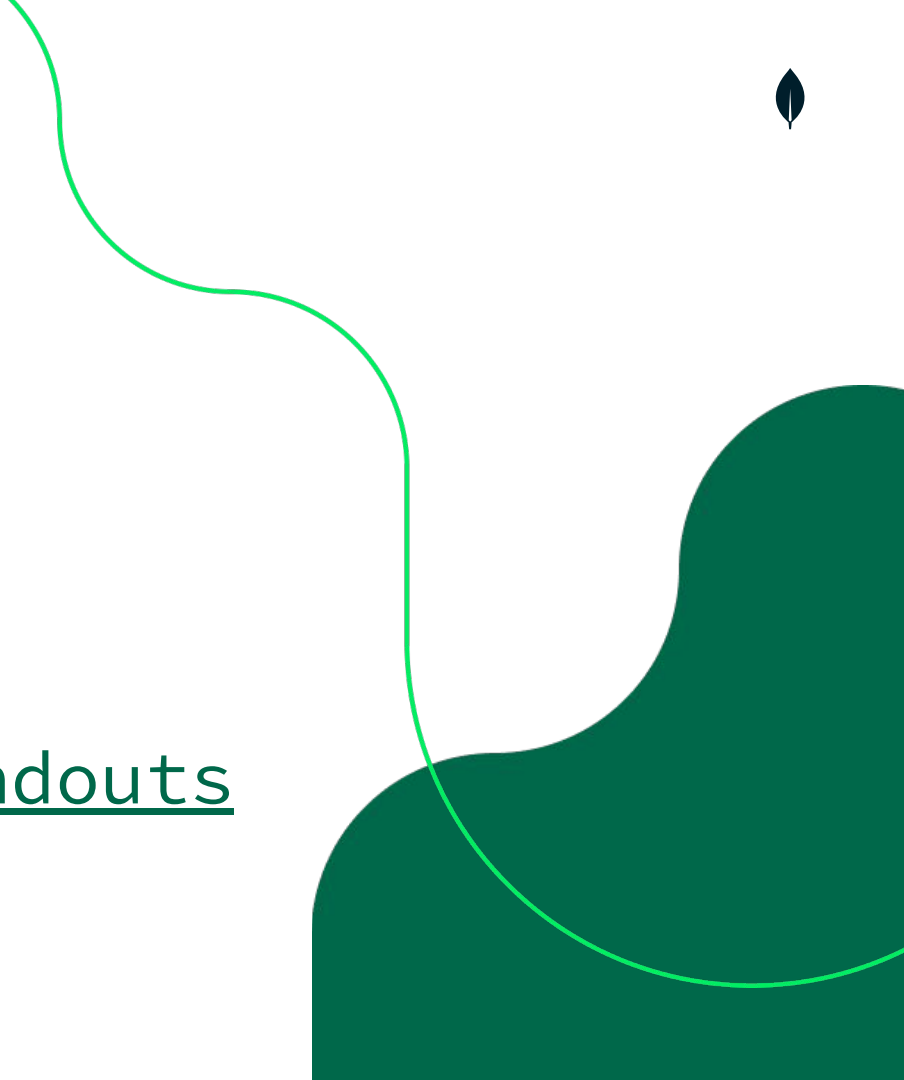
Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN

Entities

Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN



Developer Day Workbook

mdb.link/virtual-handouts



Exercise 1.

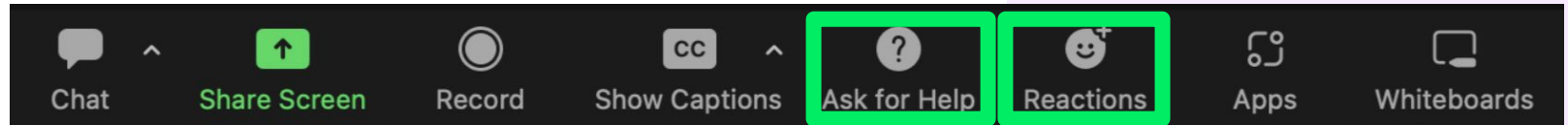
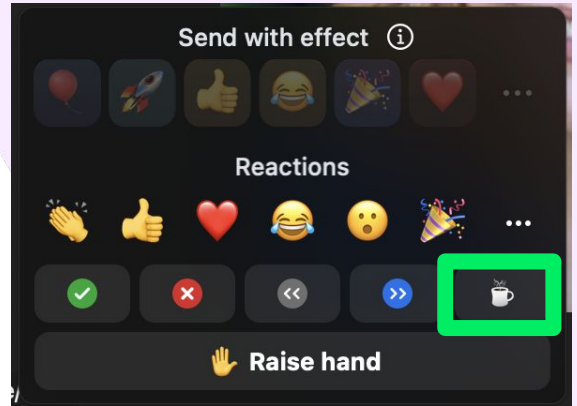
Identify the entities

**Unsure? Need more
information?
Ask the instructors!**

- What are the main entities used in this system that should be stored in the database?
- What are the properties of these entities? For example, every reservation has a due date, every review — a score, every book — a title.
- Don't normalize the data just yet.
- Don't relate the entities yet. We'll do this later.
- Is there a natural unique identifier that you can use for some entity?



Zoom breakout rooms





Entities

Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN
User	Name, address, role (reader or librarian)	Email/ library card number
Author	Name, aliases	
Reserved	Book, user, reservation date, expiration date	
Borrowed	Book, user, borrowed date, due date	
Review	Review text, rating, reviewer name, book	



Entities

Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN
User	Name, address, role (reader or librarian)	Email/ library card number
Author	Name, aliases	
Reserved	Book, user, reservation date, expiration date	
Borrowed	Book, user, borrowed date, due date	
Review	Review text, rating, reviewer name, book	
Volumes	Book, user, status	



Entities

Which entities would be part of this application, and what are some of the properties? Can you identify a unique identifier?

Entity Name	Properties	Unique Identifier
Book	Title, author, genre, cover image ...	ISBN
User	Name, address, role (reader or librarian)	Email/ library card number
Author	Name, aliases	
Reserved	Book, reservation date, expiration date	

Author:
A book property or a separate entity?

Functional Requirements



The system should allow **readers** to:

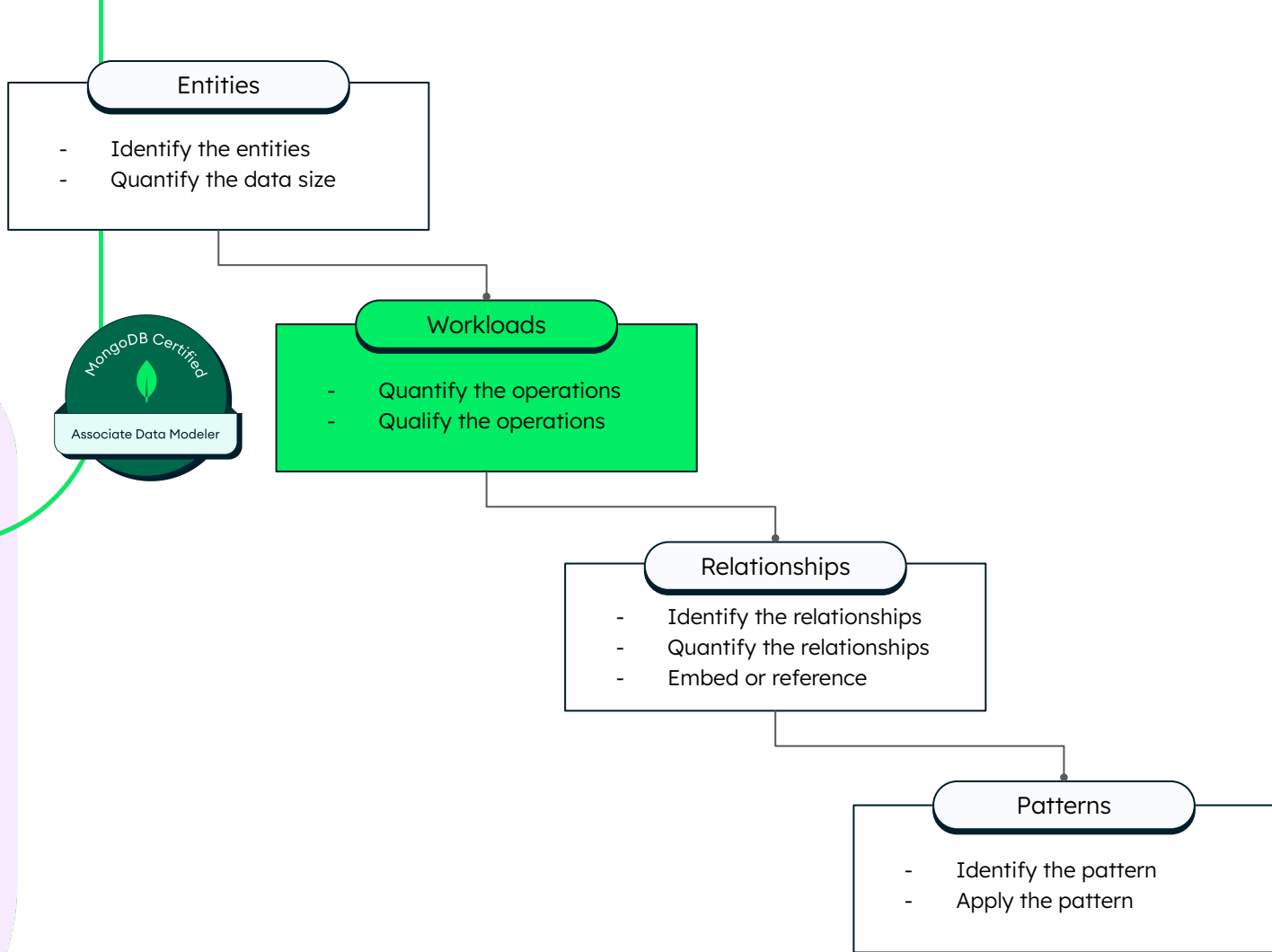
- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, synopsis — available physical copies, and reviews from readers.
- R3. Make a book reservation if there are physical copies available. Book reservations expire after 12 hours.
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.**

Authors:

Queried separately from books
=> should be a separate entity



Answer the
question in
Slido!



Step 2.

Identify the workloads

- Do not underestimate this step. This will help you decide what to optimize for.
- Will the operations be read or write?
- What data is used by each operation?
- How many requests do you expect per day?



Workloads



What are the workloads of your application?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D		
Read	Show book details	A,C,D,G		
Read	Find books	Assume same as above		
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations		
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations		
Write	Return a book	Assume all borrowed books are returned		
Write	Review for a book	Assume $\frac{1}{3}$ of borrowed books		

Workloads



What are the workloads of your application?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D		
Read	Show book details	A,C,D,G		
Read	Find books	Assume same as above		
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations		
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations		
Write	Return a book	Assume all borrowed books are returned		
Write	Review for a book	Assume $\frac{1}{3}$ of borrowed books		

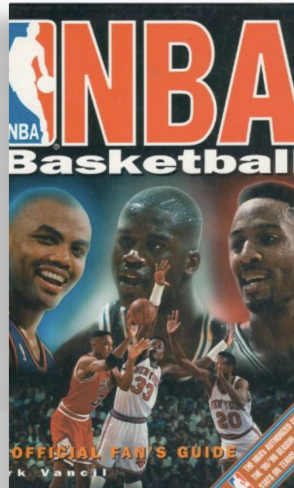


Operation

Reserve a book

API endpoint

POST /reservations/:bookId



NBA: The Official Fan's Guide

Mark Vancil

1 available copy

Reserve

Workloads



What are the workloads of your application?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D	?	
Read	Show book details	A,C,D,G		
Read	Find books	Assume same as above		
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations		
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations		
Write	Return a book	Assume all borrowed books are returned		
Write	Review for a book	Assume $\frac{1}{3}$ of borrowed books		

Functional requirements



The system should allow **readers** to:

- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, and synopsis — available physical copies, and reviews from other readers.
- R3. Make a book reservation if there are physical copies available. Book reservations expire after 12 hours.**
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.

The system should allow **librarians** to:

- L1. See a list of all book reservations, including their expiration time.
- L2. See a list of all borrowed books, including their due date.
- L3. Filter reservations and borrowed books by user.
- L4. Lend books to readers.
- L5. Mark books as returned.

Workloads



What are the workloads of your application?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D	Book, availability	
Read	Show book details	A,C,D,G		
Read	Find books	Assume same as above		
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations		
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations		
Write	Return a book	Assume all borrowed books are returned		
Write	Review for a book	Assume $\frac{1}{3}$ of borrowed books		

Workloads



What are the workloads of your application?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D	Book, availability	?
Read	Show book details	A,C,D,G		
Read	Find books	Assume same as above		
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations		
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations		
Write	Return a book	Assume all borrowed books are returned		
Write	Review for a book	Assume $\frac{1}{2}$ of borrowed books		



Library data

- A. **The library has 10k active readers weekly.**
- B. The library has about **250k books**.
- C. Readers will consult the library website on a **weekly basis** and borrow books **every week**.
- D. Most readers **reserve** and **borrow** up to **3 books** every week.
- E. **Reservations** can be made online, but books can only be **borrowed** on-site.
- F. Readers rarely read more than **5 book reviews**.
- G. Readers browse about **25 books** before reserving

Number of “show a book” requests =
10,000 users



Library data

- A. **The library has 10k active readers weekly.**
- B. The library has about **250k books**.
- C. **Readers will consult the library website on a weekly basis** and borrow books **every week**.
- D. Most readers **reserve** and **borrow** up to **3 books** every week.
- E. **Reservations** can be made online, but books can only be **borrowed** on-site.
- F. Readers rarely read more than **5 book reviews**.
- G. Readers browse about **25 books** before reserving.

Number of “show a book” requests =
10,000 users / 7 days
= ~1,400 users per day



Library data

- A. **The library has 10k active readers weekly.**
- B. The library has about **250k books**.
- C. **Readers will consult the library website on a weekly basis,** and borrow books **every week.**
- D. **Most readers reserve and borrow up to 3 books every week.**
- E. **Reservations** can be made online, but books can only be **borrowed** on-site.
- F. Readers rarely read more than **5 book reviews.**
- G. Readers browse about **25 books** before reserving

Number of “show a book” requests =
10,000 users / 7 days
= ~1,400 users per day * 3 books
= 4,200 reserved books per day

Workloads



What are the workloads of your application?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D	Book, availability	4.2K / day
Read	Show book details	A,C,D,G		
Read	Find books	Assume same as above		
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations		
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations		
Write	Return a book	Assume all borrowed books are returned		
Write	Review for a book	Assume $\frac{1}{4}$ of borrowed books		

Exercise 2.

Identify the workloads

**Unsure? Need more information?
Ask the instructors!**

- Do not underestimate this step. This will help you decide what to optimize for.
- Will the operations be read or write?
- What data is used by each operation?
- How many requests do you expect per day?

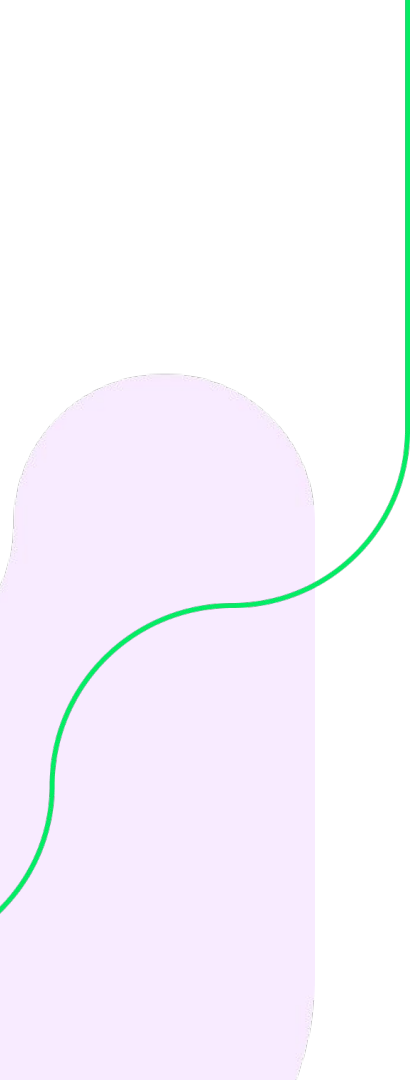


Workloads

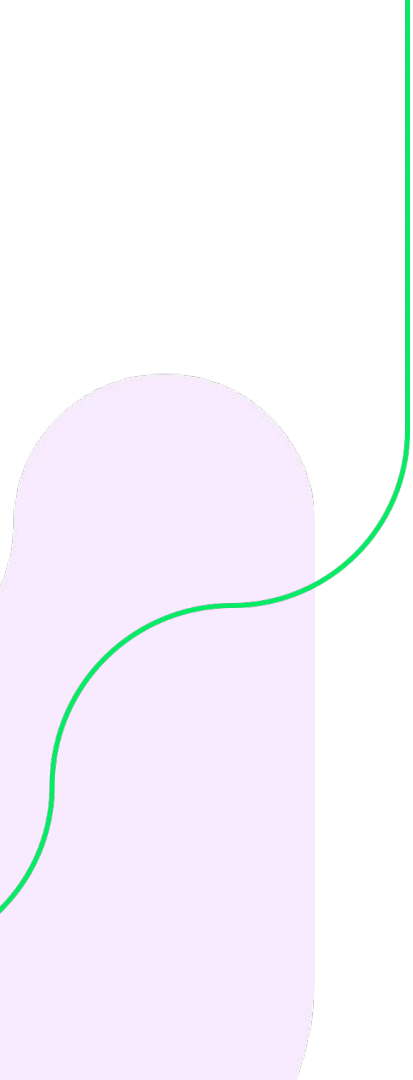


Which are the most critical workloads in your application? How critical are they?

Type	Operation	Library Data	Information	Frequency
Write	Reserve a book	A,C,D	Book, availability	4.2K / day
Read	Show book details	A,C,D,G	Book title, cover, author name, synopsis, availability, 5 reviews	100K / day
Read	Find books	Assume same as above	Book title, cover, author name	100K / day
Read	Get reservations and borrowed books by user	Assume $\frac{1}{3}$ of reservations	Reservations by user, borrowed books by user	1.4k / day
Write	Borrow a book	Assume $\frac{1}{3}$ of reservations	Borrowed book document	1.4k / day
Write	Return a book	Assume all borrowed books are returned	Borrowed book document	1.4k / day
Write	Review for a book	Assume $\frac{1}{5}$ of borrowed books	Review	280 / day



Is the workload
read-heavy or
write-heavy?



Is the workload
read-heavy or
write-heavy?

Performance Requirements



- P1. The book details page that shows book availability and recent reviews should load in less than **300ms**.
- P2. Readers should be able to reserve a book in less than **3 seconds**.
- P3. Librarians should be able to lend a book or mark a book as returned in less than **3 seconds**.
- P4. Readers should be able to see the list with all of their reservations and borrowed books in less than **500ms**.

Performance Requirements



P1. The book details page that shows book availability and recent reviews should load in less than **300ms**.

P2. Readers should be able to reserve a book in less than **3 seconds**.

P3. Librarians should be able to lend a book or mark a book as returned in less than **3 seconds**.

P4. Readers should be able to see the list with all of their reservations and borrowed books in less than **500ms**.

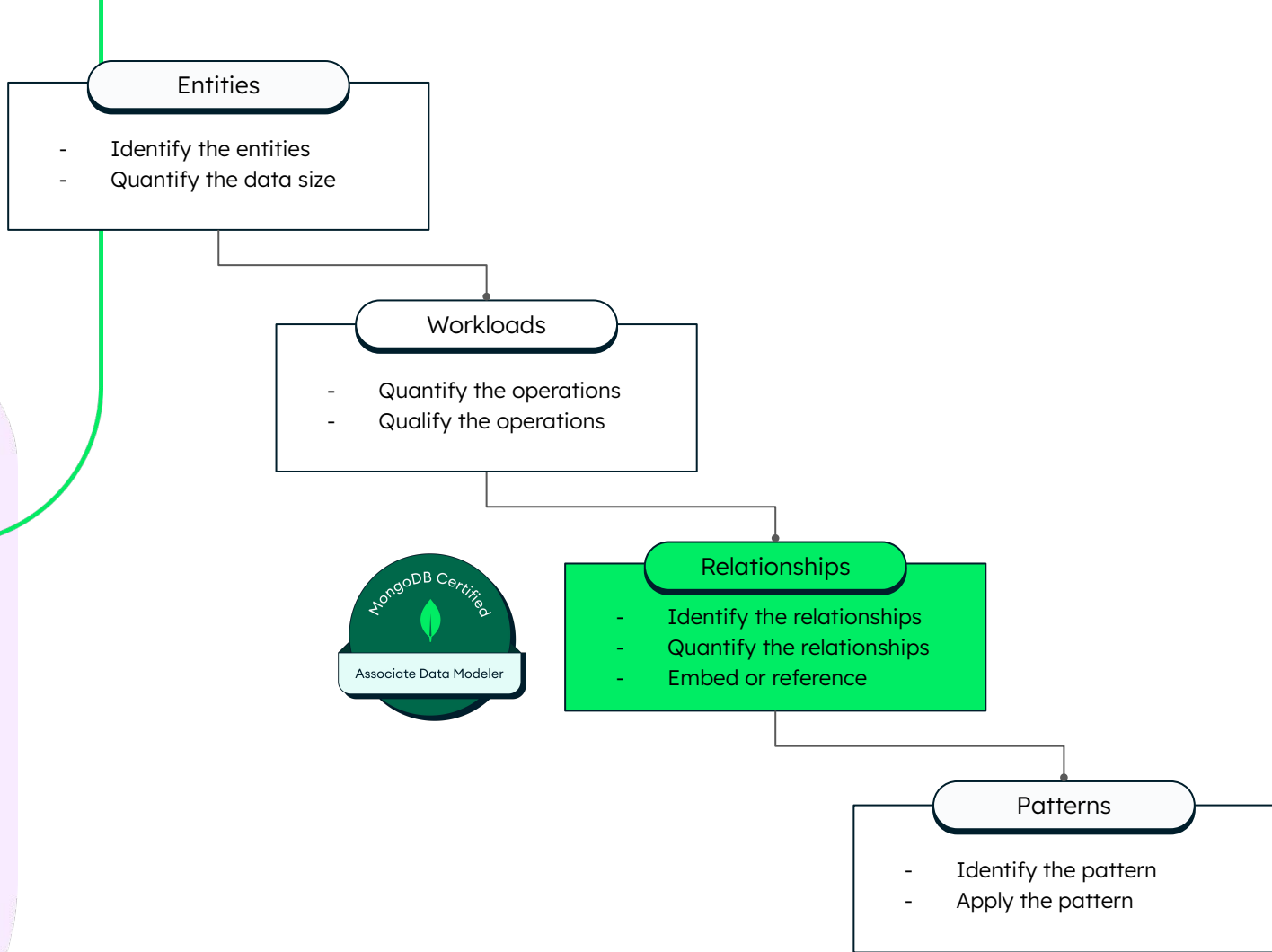


To meet the performance requirements, we must optimize the database schema for:

Fetching book details

+ book availability

+ recent reviews



Step 3.

Identify the relationships

- Look at the list of entities and identify possible relationships between them.
- Determine the type and the cardinality of the relationships.
- Decide on embedding or referencing.



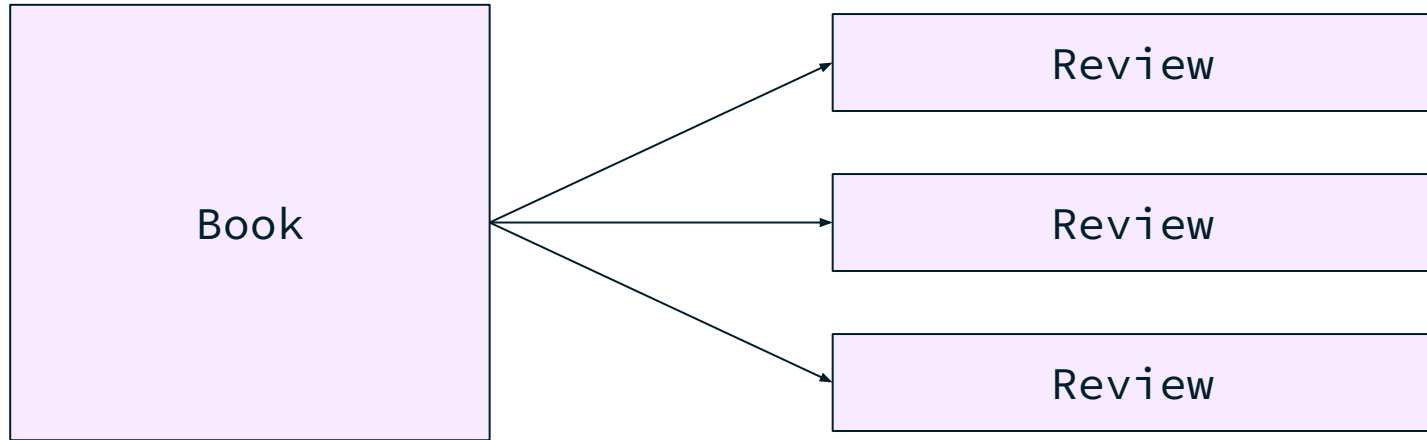


One-to-One (1-1)



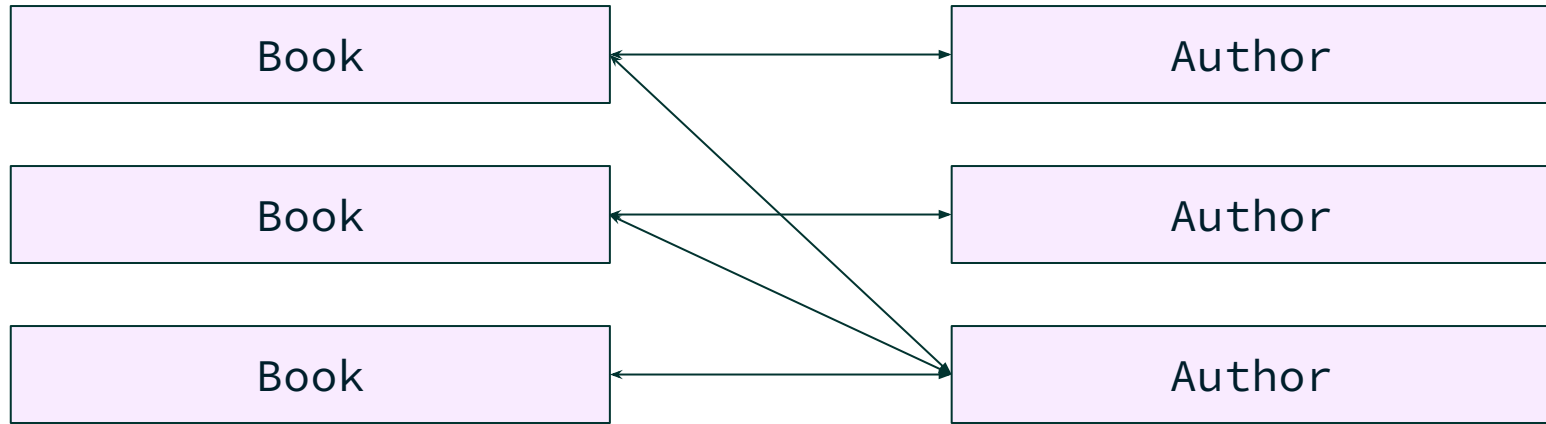


One-to-Many (1-N)





Many-to-Many (N-N)





Embedding vs Referencing

Embedding on the “one” side



Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99,  
  author: "Aldous Huxley",  
  reviews: [  
    { user: "John", text: "Cool." },  
    { user: "Jane", text: "It's ok." },  
    { user: "Mike", text: "Super!" }  
  ]  
}
```



Embedding on the “many” side

Review document 1

```
{
  _id: 1,
  user: "John",
  text: "Cool.",
  book: {
    title: "Brave New World",
    category: "books",
    price: 14.99,
    author: "Aldous Huxley"
  }
}
```

Review document 2

```
{
  _id: 2,
  user: "Jane",
  text: "It's ok.",
  book: {
    title: "Brave New World",
    category: "books",
    price: 14.99,
    author: "Aldous Huxley"
  }
}
```

Review document 3

```
{
  _id: 3,
  user: "Mike",
  text: "Super!",
  book: {
    title: "Brave New World",
    category: "books",
    price: 14.99,
    author: "Aldous Huxley"
  }
}
```

Referencing on the “one” side



Book document

```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  author: "Aldous Huxley",
  reviews: [
    1,
    2,
    3
  ]
}
```

Review document 1

```
{
  _id: 1,
  user: "John",
  text: "Cool."
}
```

Review document 2

```
{
  _id: 2,
  user: "Jane",
  text: "It's ok."
}
```

Review document 3

```
{
  _id: 3,
  user: "Mike",
  text: "Super!"
}
```


Referencing on the “many” side



Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99,  
  author: "Aldous Huxley"  
}
```

Review document 1

```
{  
  _id: 1,  
  user: "John",  
  text: "Cool.",  
  book_id: "9781602833364"  
}
```

Review document 2

```
{  
  _id: 2,  
  user: "Jane",  
  text: "It's ok.",  
  book_id: "9781602833364"  
}
```

Review document 3

```
{  
  _id: 3,  
  user: "Mike",  
  text: "Super!",  
  book_id: "9781602833364"  
}
```



Answer the
question in
Slido!

Embedding



Single query to retrieve data



Single operation to update/delete data



Data duplication



Large documents

Referencing



No duplication



Smaller documents



Need to join data from multiple documents





Guideline Name	Question	Embed	Reference
Simplicity	Would keeping the pieces of information together lead to a simpler data model and code?	Yes	
Go Together	Do the pieces of information have a "has-a," "contains," or similar relationship?	Yes	
Query Atomicity	Does the application query the pieces of information together?	Yes	
Update Complexity	Are the pieces of information updated together?	Yes	
Archival	Should the pieces of information be archived at the same time?	Yes	
Cardinality	Is there a high cardinality (current or growing) in a "many" side of the relationship?	No	Yes
Data Duplication	Would data duplication be too complicated to manage and undesired?	No	Yes
Document Size	Would the combined size of the pieces of information take too much memory or transfer bandwidth for the application?	No	Yes
Document Growth	Would the embedded piece grow without bound?	No	Yes
Workload	Are the pieces of information written at different times in a write-heavy workload?		Yes
Individuality	For the children side of the relationship, can the pieces exist by themselves without a parent?		Yes



Modeling One-to-One (1-1)



Embedding on the more frequently queried side

Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99,  
  publisher: "HarperCollins Publishers"  
}
```



Separate and use references if the embedded document is too large

Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99,  
  additionalDetails: {  
    awards: [ ... 200+ items ],  
    ... 200+ fields  
  }  
}
```



Additional Details document

```
{  
  _id: "9781602833364#add_1",  
  awards: [ ... 200+ items ],  
  ... 200+ fields  
}
```




Modeling Many-to-Many (N-N)

Referencing on the more frequently queried side



Book document 1

```
{
  title: "The Talisman",
  authors: [ 1, 2 ]
}
```

Book document 2

```
{
  title: "It",
  authors: [ 1 ]
}
```

Book document 3

```
{
  title: "Floating Dragon",
  authors: [ 2 ]
}
```

Author document 1

```
{
  _id: 1,
  name: "Stephen King"
}
```

Author document 2

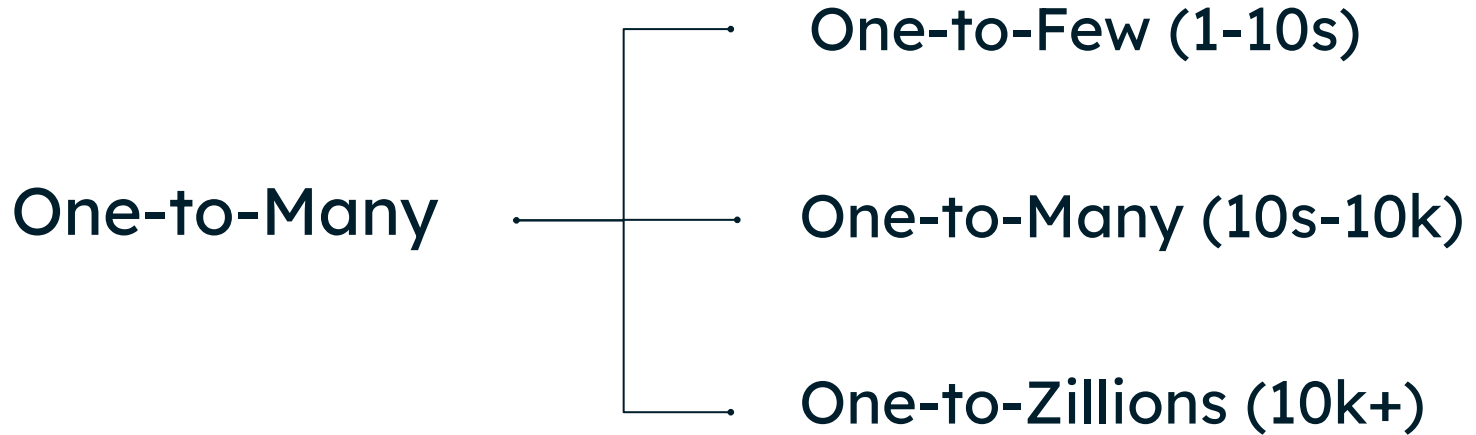
```
{
  _id: 2,
  name: "Peter Straub"
}
```



Modeling One-to-Many (1-N)



Cardinality

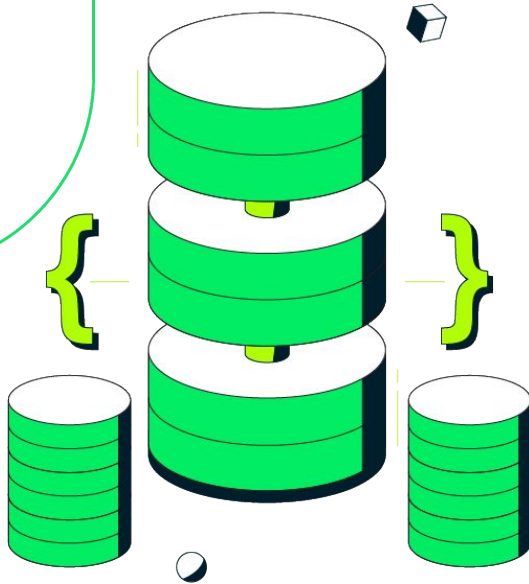




One-to-Few

Usually accessed together
and belong to one entity

- Book — Genres
- User — Addresses



One-to-Few

Embed on the “one” side



Information is accessed together with one query



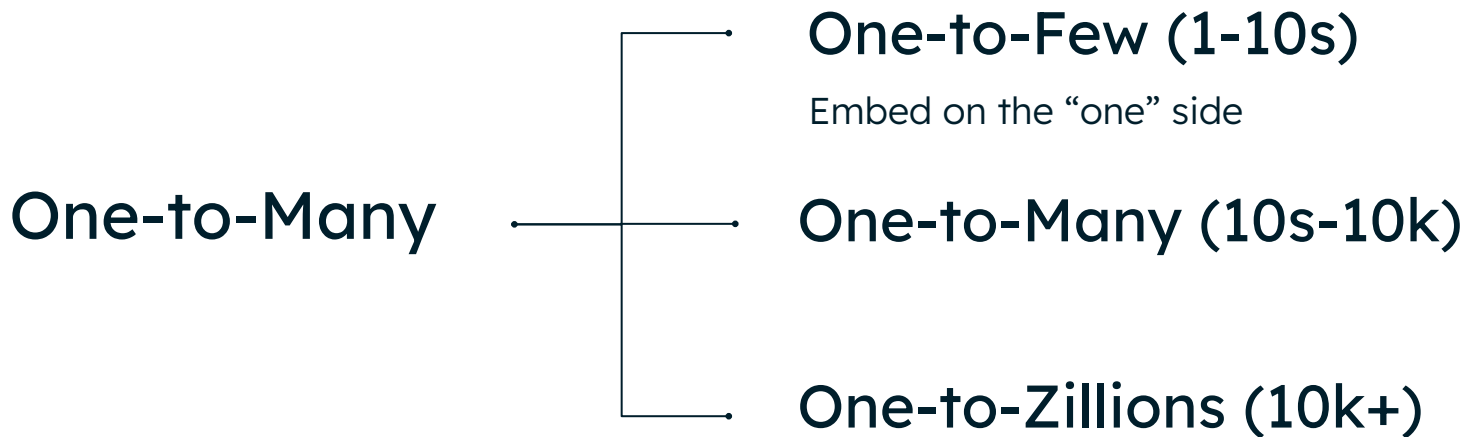
Embedded documents aren't standalone entities

Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  genres: [  
    "dystopian",  
    "science fiction",  
    "novel"  
  ]  
}
```



Cardinality

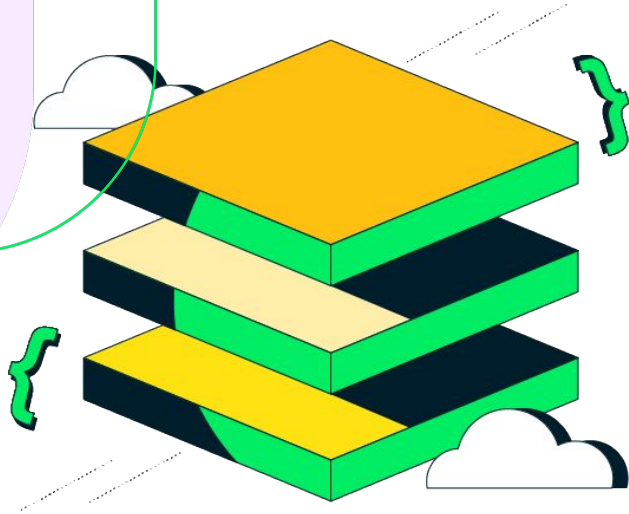




One-to-Zillions

Zillions means a *possibly* very large number of connected items

- Best-selling book — Readers





Book document

```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  readers: [
    {
      date: "2024-1-15",
      book_id: "9781602833364",
      user_id: ObjectId("...")
    },
    ... 1M+
  ]
}
```

The “readers” array can grow without bounds.
Unbounded arrays are an anti-pattern!



Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99,  
  reader_ids: [  
    ObjectId("659200802..."),  
    ObjectId("507f1f77b..."),  
    ObjectId("659200802..."),  
    ... 1M+  
  ]  
}
```

The “reader_ids” array is still
unbounded.
Anti-pattern!



Reader document 1

```
{  
  date: "2024-1-15",  
  book: {  
    _id: "9781602833364",  
    title: "Brave New World",  
    category: "books",  
    price: 14.99  
  },  
  user_id: ObjectId("...")  
}
```

Reader document 2

```
{  
  date: "2024-10-12",  
  book: {  
    _id: "9781602833364",  
    title: "Brave New World",  
    category: "books",  
    price: 14.99  
  },  
  user_id: ObjectId("...")  
}
```

Reader document 1,000,000

```
{  
  date: "2024-12-34",  
  book: {  
    _id: "9781602833364",  
    title: "Brave New World",  
    category: "books",  
    price: 14.99  
  },  
  user_id: ObjectId("...")  
}
```

Duplicated information in potentially millions of documents.



Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99  
}
```

Reader document 1

```
{  
  date: "2024-1-1",  
  book_id: "9781602833364",  
  user_id: ObjectId("507f191e810c19729de860ea")  
}
```

Reader document 2

```
{  
  date: "2024-1-15",  
  book_id: "9781602833364",  
  user_id: ObjectId("507f1f77bcf86cd799439011")  
}
```

Reader document 1,000,000

```
{  
  date: "2024-5-15",  
  book_id: "9781602833364",  
  user_id: ObjectId("6592008029c8c3e4dc76256c")  
}
```

One-to-Zillions

Reference on the “many” side



No risk of exceeding the document size limit



Be extra careful when querying the “zillions” collection

Reader document 1

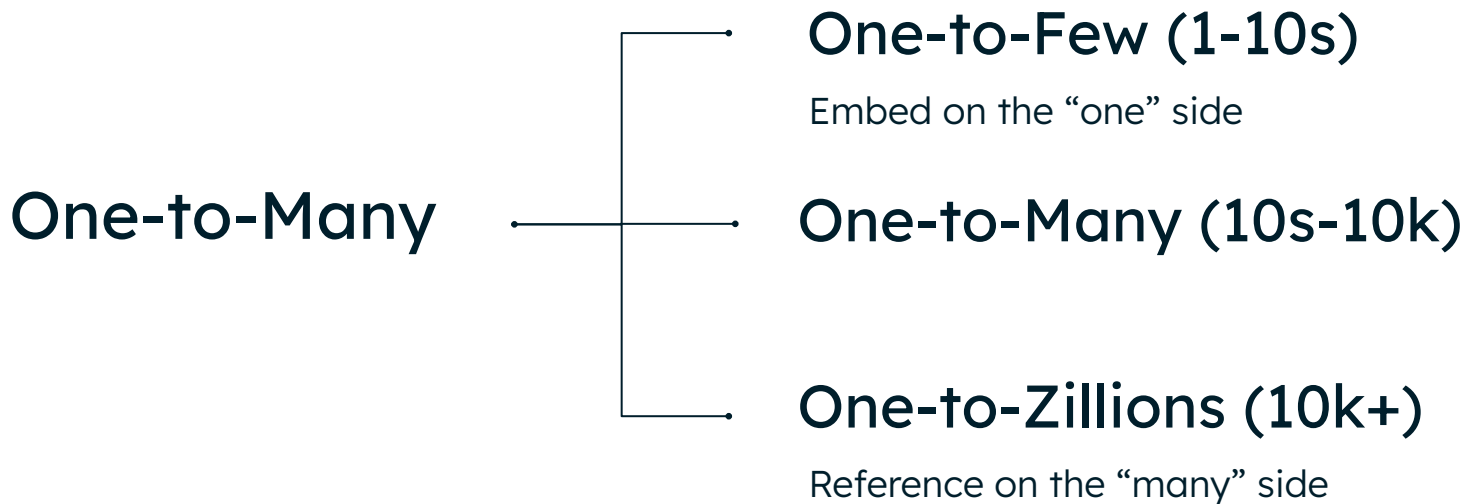
```
{  
  date: "2024-1-1",  
  book_id: "9781602833364",  
  user_id: ObjectId("507f191e810c19729de860ea")  
}
```

Reader document 2

```
{  
  date: "2024-1-15",  
  book_id: "9781602833364",  
  user_id: ObjectId("507f1f77bcf86cd799439011")  
}
```

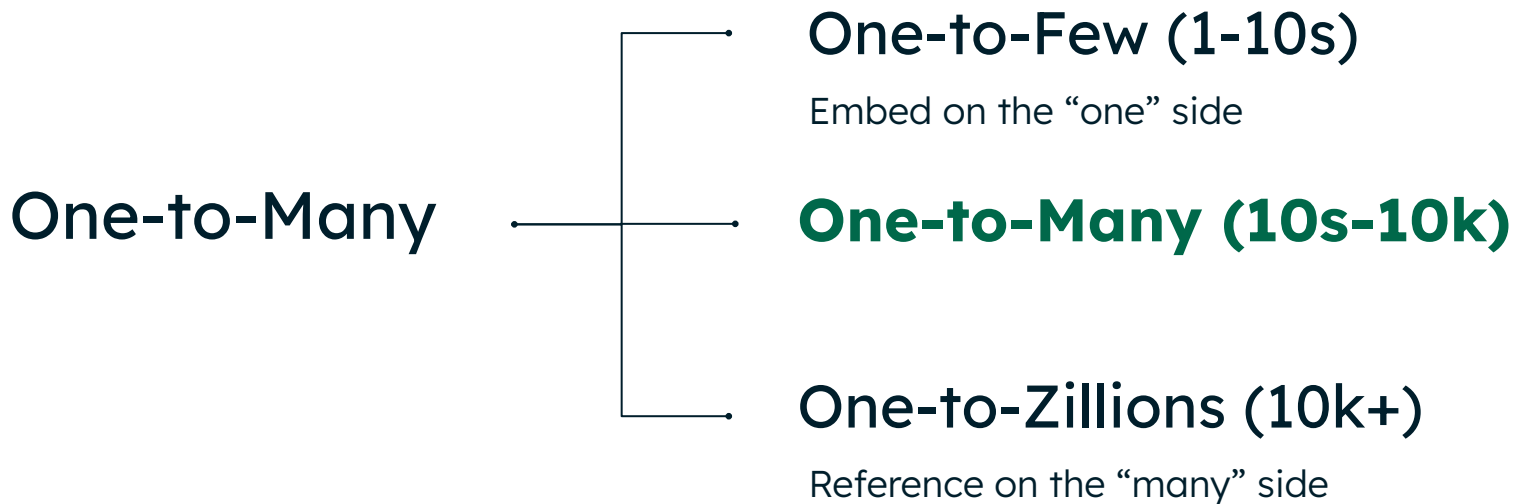


Cardinality





Cardinality





Are the entities frequently accessed together?

“Yes” - Embedding on the more often queried side

“No” - Referencing usually on the “many” side

“I’m not sure” - Referencing unless you’re looking for simplicity and you’re not afraid to migrate later



Relationships

What are the relationships between the entities? Quantify them in terms of One-to-One, One-to-Few (low cardinality), One-to-Zillions (high cardinality), Many-to-Many.

Relationship	Type	Frequently accessed separately?	Embed or Reference
Book - Author			



Relationships

What are the relationships between the entities? Quantify them in terms of One-to-One, One-to-Few (low cardinality), One-to-Zillions (high cardinality), Many-to-Many.

Relationship	Type	Frequently accessed separately?	Embed or Reference
Book - Author	Few-to-Few	?	

Functional Requirements



The system should allow **readers** to:

- R1. Browse a catalogue of library books.
- R2. View the extended details of a book — including title, author, synopsis — available physical copies, and reviews from readers.
- R3. Make a book reservation if there are physical copies available. Book reservations expire after 12 hours.
- R4. See a list of their unexpired reservations.
- R5. See a list of all books they have borrowed from the library and their due dates.
- R6. View the details for an author, including all of the books they have published.**

Authors are queried separately from books =>

Authors should be a standalone entity =>

Two collections:
Books and **Authors**

Referencing on the more frequently queried side



Book document 1

```
{
  title: "The Talisman",
  authors: [ 1, 2 ]
}
```

Book document 2

```
{
  title: "It",
  authors: [ 1 ]
}
```

Book document 3

```
{
  title: "Floating Dragon",
  authors: [ 2 ]
}
```

Author document 1

```
{
  _id: 1,
  name: "Stephen King"
}
```

Author document 2

```
{
  _id: 2,
  name: "Peter Straub"
}
```



Relationships

What are the relationships between the entities? Quantify them in terms of One-to-One, One-to-Few (low cardinality), One-to-Zillions (high cardinality), Many-to-Many.

Relationship	Type	Frequently accessed separately?	Embed or Reference
Book - Author	Few-to-Few	yes	Reference



Exercise 3.

Identify the relationships

**Unsure? Need more
information?
Ask the instructors!**

1. Determine the type and the cardinality of the listed relationships — One-to-One, One-to-Few, One-to-Zillions, Many-to-Many, etc.
2. Decide if the relationship should be modeled using embedding or referencing.

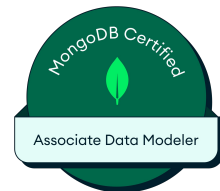
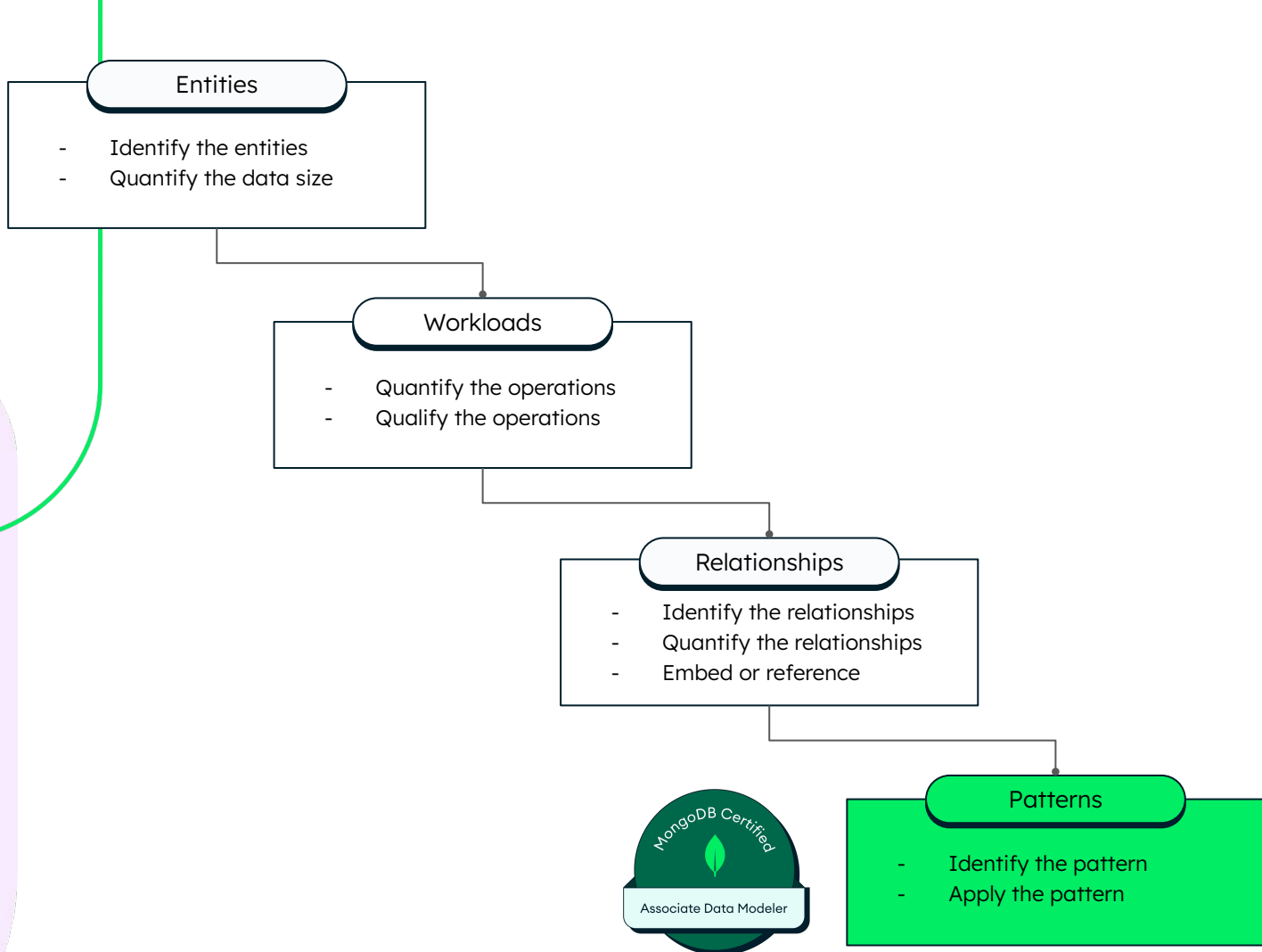


Relationships

What are the relationships between the entities? Quantify them in terms of One-to-One, One-to-Few (low cardinality), One-to-Zillions (high cardinality), Many-to-Many.

Relationship	Type	Frequently accessed separately?	Embed or Reference
Book - Author	Few-to-Few	yes	Reference
User - Borrowed books	Many-to-Many <i>Returned books are persisted as historical records</i>	yes	Reference
User - Reserved books	Many-to-Few <i>Most users reserve up to 3 books</i> <i>Past reservations aren't persisted</i>	yes	Reference
Book - Review	One-to-Many	yes	Reference

If there were no requirements to view author details, we would embed Author in Book





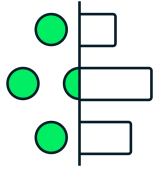
“ MongoDB design patterns are reusable solutions for many of the commonly occurring use cases encountered when designing applications that leverage persistence in MongoDB.

Daniel Coupal | Senior Staff Developer Advocate, MongoDB

[MongoDB Data Modeling and Schema Design](#)



Schema design patterns families



Computation

- Approximation
- Computed



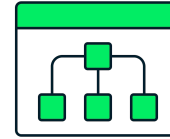
Grouping

- Bucket
- Outlier
- Preallocated



Life cycle

- Archive
- Document versioning
- Envelope
- Schema versioning



Polymorphism

- Inheritance
- Single collection

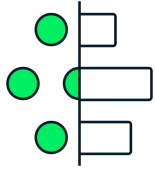


Relationships

- Attribute
- Extended reference
- Graph
- Subset
- Tree



Schema design patterns families



Computation

- Approximation
- Computed



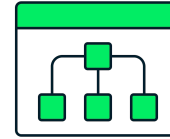
Grouping

- Bucket
- Outlier
- Preallocated



Life cycle

- Archive
- Document versioning
- Envelope
- Schema versioning



Polymorphism

- Inheritance
- Single collection



Relationships

- Attribute
- Extended reference
- Graph
- Subset
- Tree



Extended reference pattern

Minimizing the number of joins for frequent operations



REFERENCING

Get all book reservations made today

Book document 1

```
{
  _id: "0060850523",
  title: "Brave New World"
  author: "Aldous Huxley"
  genre: "..."}

```

Reservation document 1

```
{
  date: "2024-11-11",
  expires: "21:00",
  bookId: "0060850523",
  userId: 201
}
```

User document 1

```
{
  _id: 201,
  username: "mike12",
  name: "Mike Smith"
}
```

Book document 2

```
{
  _id: "1573229725",
  title: "Fingersmith"
  author: "Sarah Waters"
  genre: "..."}

```

Reservation document 2

```
{
  date: "2021-11-11",
  expires: "23:00",
  bookId: "1573229725",
  userId: 230
}
```

User document 2

```
{
  _id: 230,
  username: "jane",
  name: "Jane Doe"
}
```



```
db.getCollection('reservations').aggregate([
  {
    $match: {
      reservationDate: {
        $gte: new Date(new Date().setHours(0, 0, 0, 0)),
        $lt: new Date(new Date().setHours(23, 59, 59, 999))
      }
    }
  },
  {
    $lookup: {
      from: "books",
      localField: "bookId",
      foreignField: "_id",
      as: "book"
    }
  },
  {
    $unwind: "$book"
  },
  ...
])
```

```
...
{
  $lookup: {
    from: "users",
    localField: "userId",
    foreignField: "_id",
    as: "user"
  }
},
{
  $unwind: "$user"
},
{
  $project: {
    _id: 0,
    reservationId: "$_id",
    date: 1,
    "book.title": 1,
    "user.name": 1
  }
}
]);
```



Get all book reservations made today



No duplication



Smaller documents



Need to use 2 \$lookups



EMBEDDING

Get all book reservations made today

Reservation document

```
{
  date: "2024-11-11",
  expires: "21:00",
  book: {
    _id: "1573229725",
    title: "Fingersmith"
    author: "Sarah Waters"
    genre: "...",
    publisher: "..."
  },
  user: {
    _id: 230,
    username: "jane",
    name: "Jane Doe",
    addresses: [...]
  }
}
```



Only 1 query



Large documents



Data duplication



Unnecessary data

Extended reference pattern



Reservation document

```
{
  date: "2024-11-11",
  expires: "21:00",
  book: {
    _id: "1573229725",
    title: "Fingersmith"
    author: "Sarah Waters"
    genre: "..."
    publisher: "..."
  },
  user: {
    _id: 230,
    username: "jane",
    name: "Jane Doe",
    addresses: "Jane Doe"
  }
}
```



Reservation document

```
{
  date: "2024-11-11",
  expires: "21:00",
  book: {
    _id: "1573229725",
    title: "Fingersmith"
  },
  user: {
    _id: 230,
    name: "Jane Doe",
  }
}
```



EXTENDED REFERENCE

Get all book reservations made today

Reservation document

```
{
  date: "2024-11-11",
  expires: "21:00",
  book: {
    _id: "1573229725",
    title: "Fingersmith"
  },
  user: {
    _id: 230,
    name: "Jane Doe"
  }
}
```



Only 1 query



Small documents



No unnecessary data



Some data duplication



Extended reference pattern

- Used when data is stored in separate collections that need to be joined frequently
- Avoids a \$lookup by adding a subset of the related document, at the cost of duplicating data
- Use when you have separate collections but a frequent need for part of the related document



Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	?	?



Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.



Computed pattern

Precompute values on write to optimize read performance

Average book rating based on reviews



Book Review document 1

```
{  
  bookId: "103912X921",  
  rating: 3,  
}
```

Book Review document 2

```
{  
  bookId: "103912X921",  
  rating: 1,  
}
```

Book Rating:

$$(3 + 1) / 2 = 2$$

Calculating on every “get book” query



Book Review document 1

```
{  
  bookId: "103912X921",  
  rating: 3  
}
```

Book Review document 2

```
{  
  bookId: "103912X921",  
  rating: 1  
}
```

...

Book Review document 999999

```
{  
  bookId: "103912X921",  
  rating: 4.5  
}
```



Rating is always accurate



Querying potentially millions of documents on every request



Calculating the same value over and over



Slower “get book” queries

Computed field in the book document



Book document

```
{  
  _id: "9781602833364",  
  averageRating: 3.75,  
  title: "Brave New World",  
  category: "books",  
  price: 14.99  
}
```



Avoid recomputing on every read



Control over recomputing period:
on every write, once a day, other



Staleness of the computed field

Computed pattern

- Precompute and store values, offsetting the computation on write instead of read
- Addresses the case where computational tasks are performed frequently on data retrieval
- Use on high read-to-write ratio on documents where computation is needed





Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	?	?



Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.



Subset pattern

Reduce joins and get smaller documents
by embedding a subset in an array

Modeling “book-readers” relationship



Book document

```
{  
  _id: "9781602833364",  
  title: "Brave New World",  
  category: "books",  
  price: 14.99  
}
```

Reader document 1

```
{  
  date: "2024-1-1",  
  book_id: "9781602833364",  
  user_id: ObjectId("507f191e810c19729de860ea")  
}
```

Reader document 2

```
{  
  date: "2024-1-15",  
  book_id: "9781602833364",  
  user_id: ObjectId("507f1f77bcf86cd799439011")  
}
```

Reader document 1,000,000

```
{  
  date: "2024-5-15",  
  book_id: "9781602833364",  
  user_id: ObjectId("6592008029c8c3e4dc76256c")  
}
```

Store a subset of the readers in the book



Book document

```
{
  _id: "9781602833364",
  title: "Brave New World",
  category: "books",
  price: 14.99,
  latest_readers: [
    { _id: 123, name: "Jack" },
    { _id: 234, name: "John" },
    { _id: 345, name: "Paige" }
  ]
}
```



Avoid joining data



Control over the stored subset



Additional code to maintain the subset



Subset pattern

- Splits a collection in two parts, separating frequently used data from less frequently accessed data
- Tackles the problem of arrays that would contain too many entries
- Use when you only need a small portion of the entire data, and you can query the larger collection only when needed



Patterns

Can you identify some design patterns that can be applied?

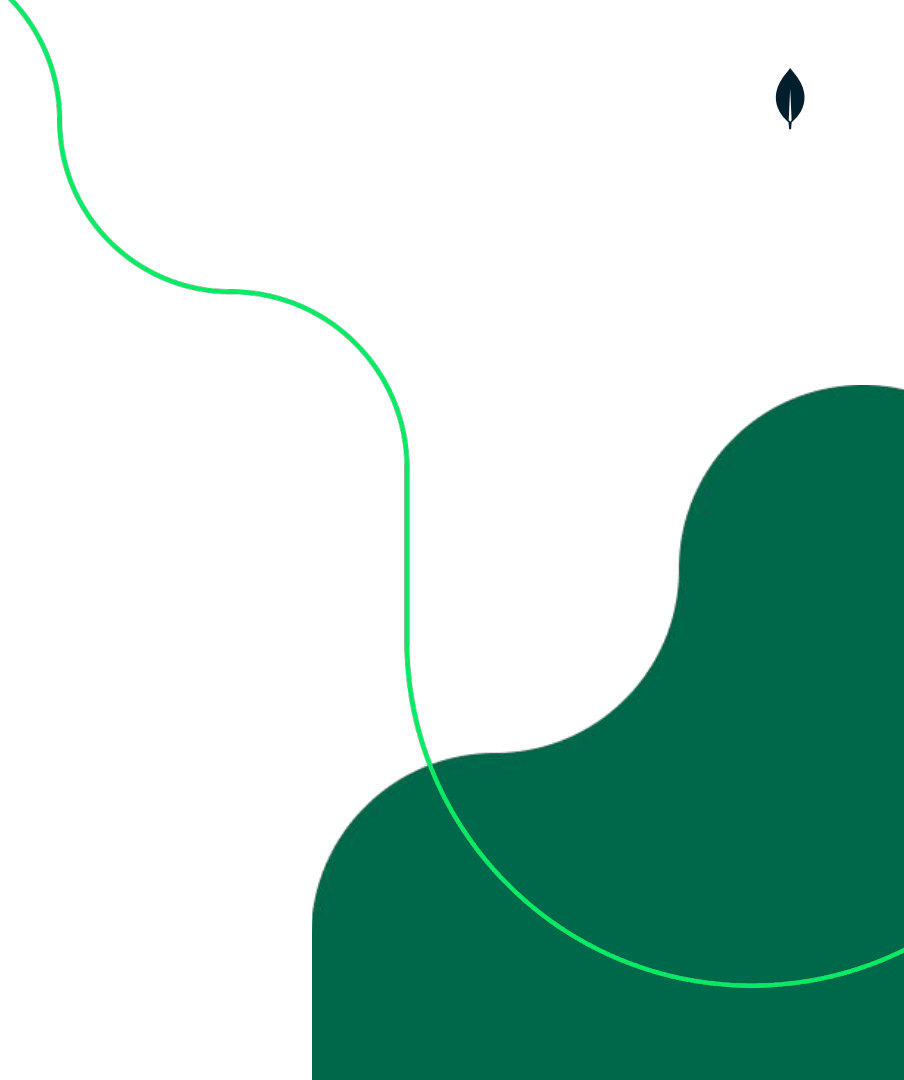
Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.
Subset	?	?



Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.
Subset	Reviews	Keep a subset of the latest reviews in the book document.



Inheritance pattern

Simplify the retrieval of “similar” data
by storing it in a polymorphous collection

Get the five most expensive products across three product collections



Electronics document

```
{  
  type: "electronics"  
  brand: String  
  model: String  
}
```

Clothing document

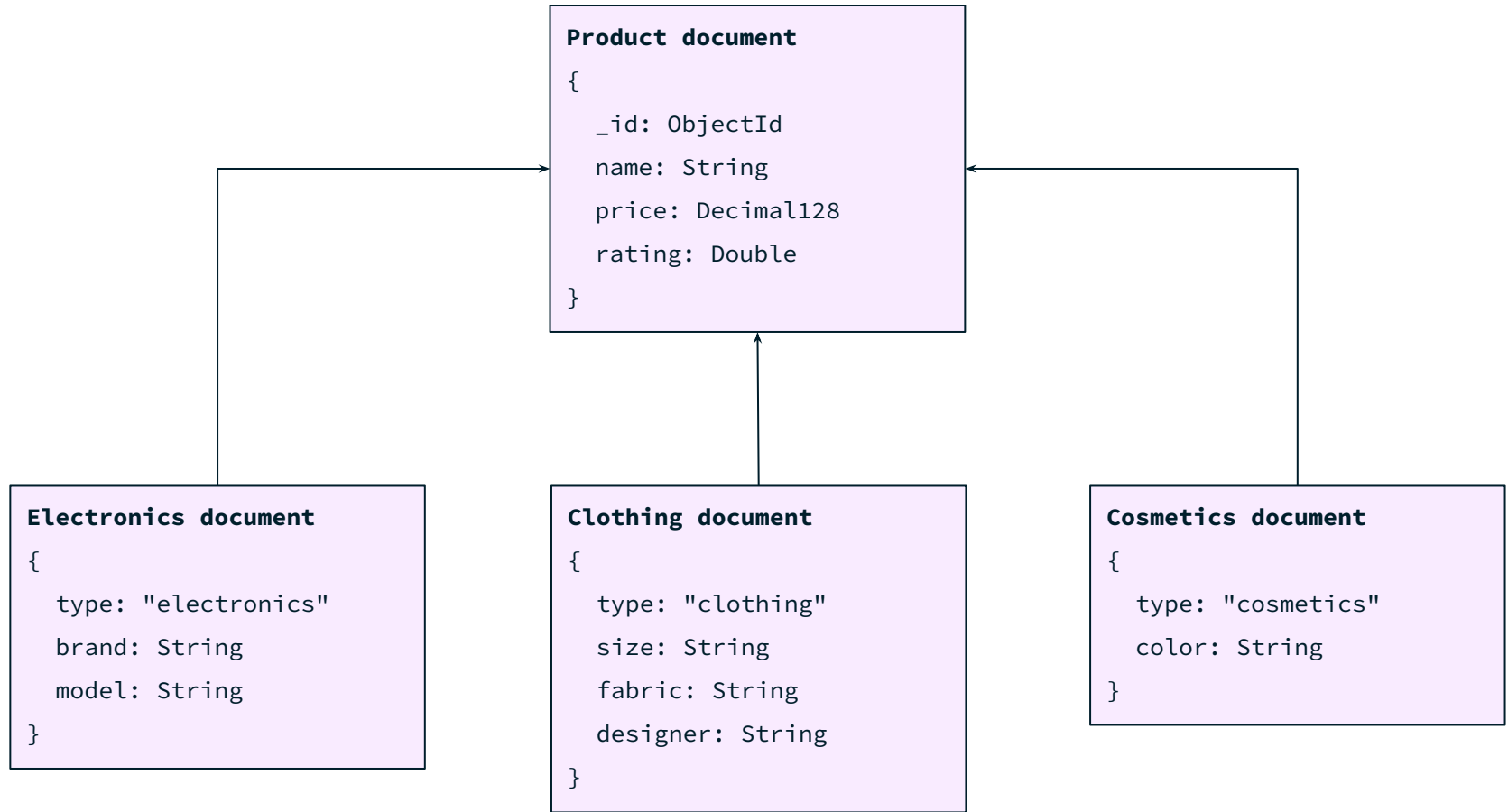
```
{  
  type: "clothing"  
  size: String  
  fabric: String  
  designer: String  
}
```

Cosmetics document

```
{  
  type: "cosmetics"  
  color: String  
}
```

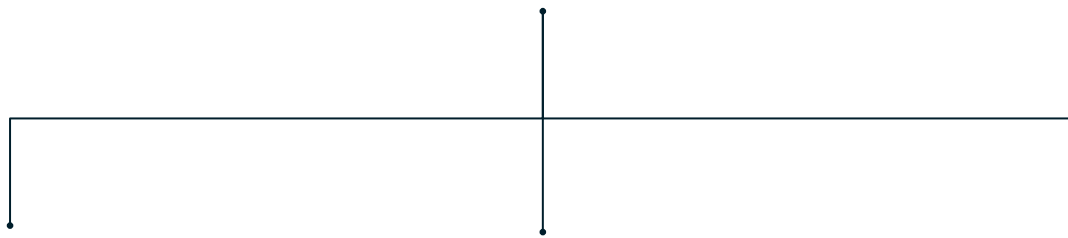


```
db.getCollection("electronics").aggregate([
  {
    "$unionWith": {
      "coll": "cosmetics"
    }
  },
  {
    "$unionWith": {
      "coll": "clothing"
    }
  },
  {
    "$sort": {
      "price": -1
    }
  },
  {
    "$limit": 5
  }
])
```





Products collection



Electronics document

```
{
  type: "electronics"
  brand: String
  model: String
}
```

Clothing document

```
{
  type: "clothing"
  size: String
  fabric: String
  designer: String
}
```

Cosmetics document

```
{
  type: "cosmetics"
  color: String
}
```



```
db.getCollection("products").aggregate([
  {
    "$sort": {
      "price": -1
    }
  },
  {
    "$limit": 5
  }
])
```


Inheritance

- Allows similar types of data to be stored in the same collection
- Addresses the issue with documents that are similar but still have distinct features; allows simpler retrieval of data as it can coexist
- Use when the application contains similar documents that need to be queried together





Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.
Subset	Reviews	Keep a subset of the latest reviews in the book document.
Inheritance	?	?



Patterns

Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.
Subset	Reviews	Keep a subset of the latest reviews in the book document.
Inheritance	Borrowed books and reservations	They are similar in shape and retrieved together. We need both to calculate the book availability.



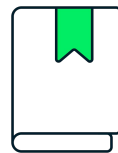
Attribute pattern

Simplify indexing and enable non-deterministic field names



Electronics document

```
type: "electronics",  
brand: "Apple",  
model: "14 Pro Max",  
RAM: "6GB"
```



Book document

```
type: "book",  
size: "pocket",  
pages: 304,  
edition: "1st"
```



```
db.products.createIndex({ "type": 1 })
```

```
db.products.createIndex({ "brand": 1 })
```

```
db.products.createIndex({ "model": 1 })
```

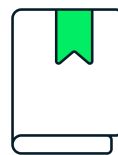
```
db.products.createIndex({ "size": 1 })
```

...



Electronics document

```
type: "electronics",  
brand: "Apple",  
model: "14 Pro Max",  
RAM: "6GB"
```



Book document

```
type: "book",  
size: "pocket",  
pages: 304,  
edition: "1st"
```

Electronics document

```
type: "electronics",  
attributes: [  
  { key: "brand", value: "Apple" },  
  { key: "model", value: "14 Pro Max" },  
  { key: "RAM", value: "6GB" }  
]
```

Book document

```
type: "book",  
attributes: [  
  { key: "size", value: "pocket" },  
  { key: pages, value: 304 },  
  { key: edition, value: "1st" }  
]
```

```
db.products.createIndex({ "attributes.key": 1, "attributes.value": 1 })
```

Attribute pattern

- Transforms subset of information into arrays of key-value pairs within documents
- Simplifies indexing for a document and enables non-deterministic field name
- Use when dealing with collections of documents with diverse and unpredictable values



Patterns



Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.
Subset	Reviews	Keep a subset of the latest reviews in the book document.
Inheritance	Borrowed books and reservations	They are similar in shape and retrieved together. We need both to calculate the book availability.
Attribute	?	?

Patterns



Can you identify some design patterns that can be applied?

Pattern	Entities	Reason
Extended Reference	Authors	Copy some fields from the author document in the book document. Fetch all required book details with a single query.
Computed	Available books	Maintain the inventory as a computed field in the book document rather than computing on every request.
Subset	Reviews	Keep a subset of the latest reviews in the book document.
Inheritance	Borrowed books and reservations	They are similar in shape and retrieved together. We need both to calculate the book availability.
Attribute	Books	Reduce the indexing needs and account for rare and unpredictable fields.



Schema versioning pattern

Make your schema adaptable to changes



Schema versioning

```
title: "Brave New World",  
category: "books",  
price: 14.99,  
author: "Aldous Huxley"
```

New field

```
title: "Brave New World",  
category: "books",  
price: 14.99,  
author: "Aldous Huxley"  
publisher: "Langdon",  
schemaVersion: 2
```



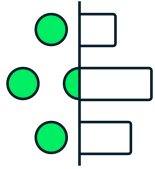
Schema versioning

- Involves adapting your documents to new schema version as the application evolves
- Can be useful when there is a big change in the schema between versions of the application
- Use when the schema has to undergo large changes but where downtime is not an option





Schema design patterns families



Computation

- Approximation
- Computed



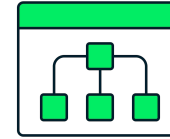
Grouping

- Bucket
- Outlier
- Preallocated



Life cycle

- Archive
- Document versioning
- Envelope
- Schema versioning



Polymorphism

- Inheritance
- Single collection



Relationships

- Attribute
- Extended reference
- Graph
- Subset
- Tree



Building with Patterns: Blog Post Series





Library system database schema



```
User {  
  _id: ObjectId;  
  name: string;  
  isAdmin: boolean;  
}
```

```
Review {  
  _id: ObjectId;  
  text: string;  
  name: string;  
  rating: number;  
  timestamp: number;  
  // Reference to the book collection.  
  bookId: string;  
}
```




```
Author {  
  _id: ObjectId  
  name: string;  
  aliases: Array<string>;  
  bio: string;  
  // Array of ISBNs of books written by this author.  
  // Reference to the books collection.  
  books: Array<string>;  
}
```



```
Book {  
  // ISBN (International Standard Book  
  Number) of the book. - unique  
  _id: string;  
  title: string;  
  cover: string;  
  genres: Array<string>;  
  synopsis: string;  
  // Number of books in total.  
  totalInventory: number;  
  // Number of books currently available.  
  // This field is computed  
  available: number;  
  ...
```

```
  // Array of author references following the  
  extended reference pattern.  
  authors: Array<{_id: ObjectId; name: string}>;  
  // Array of attributes following the attribute  
  pattern (key-value pairs).  
  attributes: Array<{key: string; value: string}>;  
  // Array of reviews following the subset pattern.  
  reviews: Array<Review>;  
}
```



Reservations / Borrowed – Inheritance

```
IssueDetailBase {  
    // Identifier with the format 'userId(B|R)objectId' to optimize querying by  
    // user.  
    _id: string;  
    // Type of record: 'borrowedBook' or 'reservation'.  
    recordType: string;  
    // Reference to the book collection following the extended reference pattern.  
    book: ReservationBook;  
    // Reference to the user collection following the extended reference pattern.  
    user: ReservationUser;  
}
```



Reservations / Borrowed – Inheritance

```
Reservation extends IssueDetailBase {  
    /**  
    * Date when the reservation expires.  
    * TTL index applied to this field to automatically remove  
    the reservation.  
    */  
    expirationDate: Date;  
}
```



Reservations / Borrowed – Inheritance

```
BorrowedBook extends IssueDetailBase {  
    borrowDate: Date;  
    dueDate: Date;  
    returnedDate: Date;  
    returned: boolean;  
}
```

Schema validation





```
User {  
  _id: ObjectId;  
  name: string;  
  isAdmin: boolean;  
}
```



```
const userSchema = {
  bsonType: 'object',
  required: ['name', 'isAdmin'],
  properties: {
    name: {
      bsonType: 'string',
      description: 'must be a string and is required'
    },
    isAdmin: {
      bsonType: 'bool',
      description: 'must be a boolean and is required'
    }
  }
};
```




```
db.command({  
  collMod: 'users',  
  validator: {  
    $jsonSchema: userSchema  
  },  
  validationLevel: 'strict',  
  validationAction: 'error'  
});
```



Indexing



What are indexes?

Data structures that provide faster access to specific data

Speed up queries, reduce disk I/O

Write operations require indexes to be updated

MongoDB only uses *one* index to fulfill queries*

* expect in a few special cases



Best practices

Every query must use an index

Drop unused or unnecessary indexes

Follow the ESR rule for compound indexes



ESR rule (Equality, Sort, Range)

[alexbevi.com/blog/2020/05/16/
optimizing-mongodb-compound-indexes-the-equality-sort-range-esr-rule/](https://alexbevi.com/blog/2020/05/16/optimizing-mongodb-compound-indexes-the-equality-sort-range-esr-rule/)



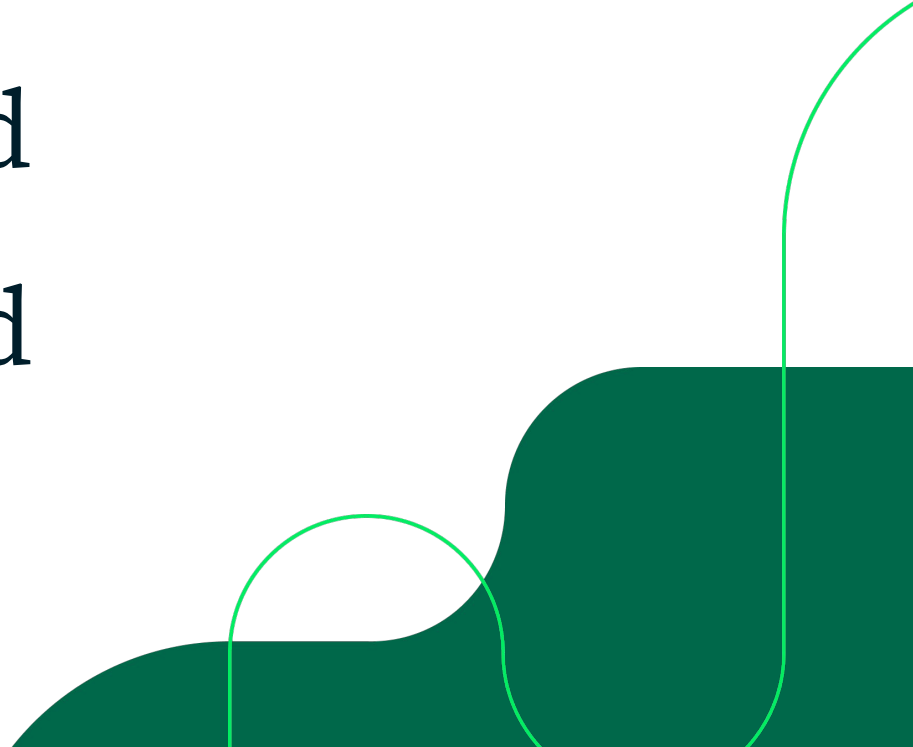
“

The ESR rule focuses on
creating a **compound index**
in the most efficient way.





“
Equality field first
Sort field second
Range field third





```
cars.find({  
  manufacturer: 'Ford',  
  cost: { $gt: 15000 }  
}).sort({ model: 1 });
```

```
cars.createIndex({  
  manufacturer: 1, // 1. Equality  
  model: 1,       // 2. Sort  
  cost: 1         // 3. Range  
});
```




Explain plan results

```
inventory.find(  
  { quantity: { $gte: 100, $lte: 200 } }  
)explain();
```



```
queryPlanner: {  
  winningPlan: {  
    queryPlan: {  
      stage: 'COLLSCAN',  
    }  
  }  
},  
executionStats: {  
  executionSuccess: true,  
  nReturned: 3,  
  executionTimeMillis: 0,  
  totalKeysExamined: 0,  
  totalDocsExamined: 10,  
  executionStages: {  
    stage: 'COLLSCAN',  

```

...



When modeling data for
MongoDB, **your workload
should drive your data model.**

Session survey

←→↺

toj11001.github.io/intro-lab/docs/intro

☆🔖📄🌐⋮

MongoDBIntro Lab

⚙️🔍 Search...

Introduction

MongoDB Cluster ▾

📖 Library Management System Dataset

👉 Get Your Connection String

Open the Application >

Schema Validation >

Indexing >

Optimize the Application >

Troubleshooting the application >

🏠 Summary

🏠 > Introduction

Introduction

Workshop goals	Get your environment ready for other hands-on labs
Prerequisites	A web browser - See Requirements
Time to complete	10-15 minutes

These labs are meant to be presented by one of our amazing instructors, but you can also do them on your own.

In the navigation bar, you will notice some icons. Here are their meanings:

Icon	Meaning
📖	Lecture material - If you're following along in an instructor-led session, they have probably covered this material already.
👉	Hands-on content - Get ready to get some work done. You should follow these steps.
👤	Advanced content - This content isn't covered during the lab, but you can check it out if you're interested in learning more.
🔧	Troubleshooting information - This content is designed to help you resolve common problems.

Requirements

To ensure you have the best experience during our sessions, please review the following minimum

Minimum hardware requirements:

Operating system:

Processor:

Memory:

Storage:

Internet connection:

Recommendations:

Other requirements:

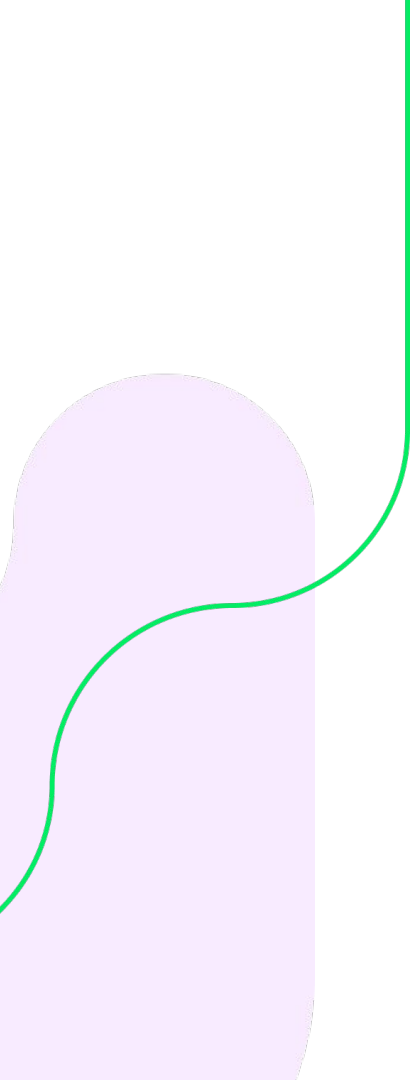
Lab



Macht die Intro Lab:

<https://toj11001.github.io/intro-lab/>

Es ist wichtig für den nächsten
Workshop

A decorative graphic on the left side of the slide. It consists of a light purple, rounded rectangular shape at the bottom, with a thin green line curving upwards from its top right corner and extending towards the top left of the slide.

What if I get
stuck?

What is MongoDB Atlas?

mongodb-developer.github.io/intro-lab/docs/mongodb-atlas/what-is-mongodb

Incognito

 **MongoDB** Intro Lab

 Search ⌘+K

Introduction

MongoDB Atlas >

Importing Data >

Open the Application >

Schema Validation >

Indexing >

Optimize the Application >

Troubleshooting the application >

Summary

Home > MongoDB Atlas > What is MongoDB Atlas?



What is MongoDB Atlas?

MongoDB comes in many different flavors. The most common is the open-source version that you can download and install on your computer. However, there is also a cloud-based version of MongoDB that you can use. This cloud offering is called MongoDB Atlas.

In this lab, you will learn how to create a MongoDB Atlas account and a cluster. You will also learn how to connect to your cluster from an application that runs in the cloud.

Why use MongoDB Atlas?

There are many reasons why you might want to use MongoDB Atlas, but most of all, it's by far the easiest way to get started with MongoDB.

On top of that, MongoDB Atlas offers a lot of features that you would otherwise have to set up yourself. For example, MongoDB Atlas offers:

Why use MongoDB Atlas?

Where is MongoDB Atlas deployed?

How much does MongoDB Atlas cost?

<<

New Associate Data Modeler Certification

PREPARE

Take the free online MongoDB Data Modeling Path. It will help you build your knowledge and prepare for the exam.

REGISTER

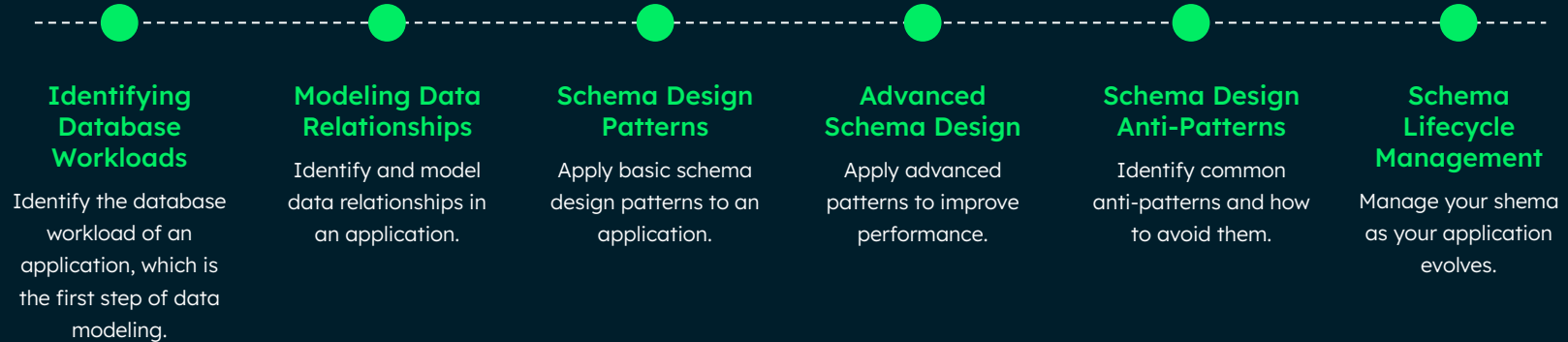
Once you complete the learning path, you'll automatically receive a 75% discount code. Use it to schedule your exam.



Start here! Register to take the Data Modeling Path.

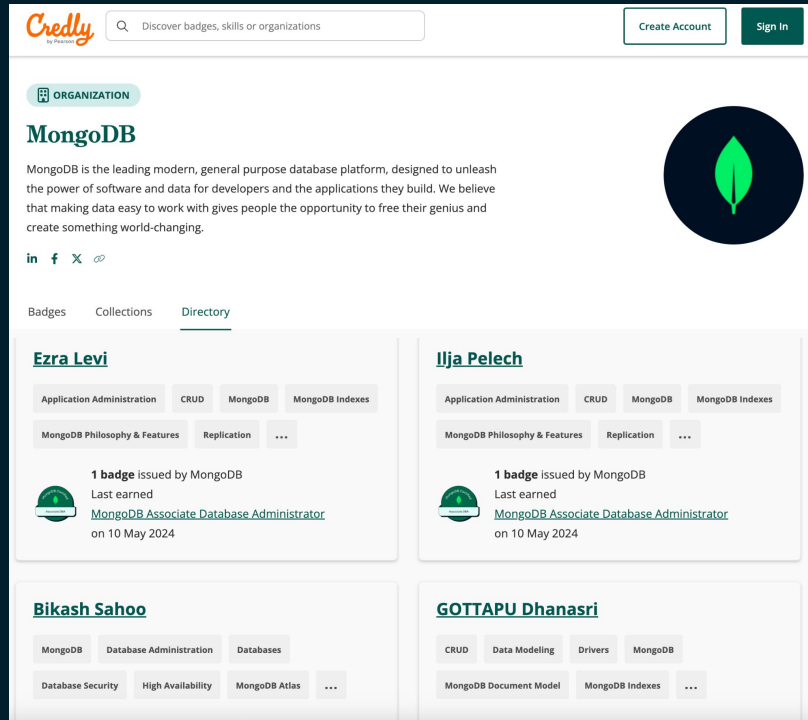


Data Modeling Learning Path



Certification Benefits

- **Formal validation:** Widely recognized certifications offer formal validation of expertise.
- **Digital badges:** Showcase achievements with a distinguished Credly badge and inclusion in the Credly Talent Directory.
- **Career advancement:** Elevate in current roles and increase appeal for future positions.



Exam Details

TEST FORMAT

70 questions, multiple choice

DELIVERY FORMAT

Online proctored

TIME ALLOTTED

105 minutes

PREREQUISITES

None

EXAM PRICE

~~\$150 USD~~ \$37.50 (with discount)

LANGUAGE

English

Questions? Contact certification@mongodb.com



Thank you!



DESIGN REVIEW

Request a Session!

Scan the QR code to request a design review for you and your team to get answers to all of your schema design questions!



mdb.link/request-design-review